

The background of the slide features a faint, stylized illustration of the Giralda tower from the University of Seville. To the left of the tower, there is a winged figure, possibly an angel or a personification of a concept, holding a staff or scepter. The entire background is rendered in a light, textured, golden-brown color.

T7 - Introducción a las BBDD y al Modelo Relacional

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA

Objetivos



Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



¿Qué es una base de datos?

Un sistema de gestión de base de datos (SGBD) es un sistema informático que:

- Implementa (principalmente) las funciones de memoria e informativa de un sistema de información.
- Almacena grandes volúmenes de datos.
- Gestiona el acceso concurrente a los datos.
- Mantiene la integridad semántica de los datos.
- Controla el acceso a los datos.

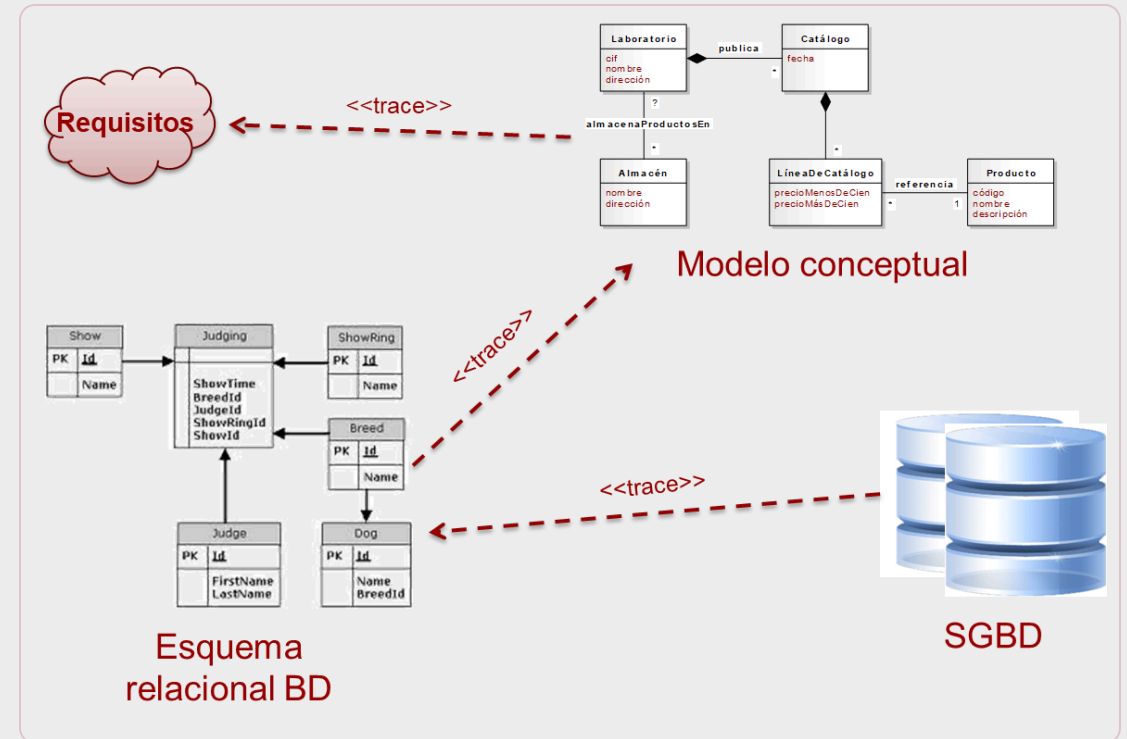


Trazabilidad

Una vez definido el **modelo conceptual**, el siguiente paso es diseñar cómo se almacenará la información.

Primero veremos qué propiedades debe tener un **modelo relacional** bien diseñado.

Después podremos estudiar cómo **transformar sistemáticamente** el modelo conceptual en relaciones.



Modelo relacional

El modelo relacional fue propuesto por **E. F. Codd** en 1970.

Su éxito se debe a tres ideas:

- Representa la información con una estructura sencilla: **relaciones**.
- Se apoya en fundamentos matemáticos: conjuntos y lógica de predicados.
- Se puede consultar de forma declarativa: indicamos qué queremos obtener, no cómo recorrer los datos.

Sistemas relacionales actuales: PostgreSQL, MySQL, MariaDB, SQLite, Oracle, SQL Server, DB2.



Tipos de bases de datos

Hoy conviven varios tipos de bases de datos. La elección depende del problema.

- **Relacionales (SQL):** datos transaccionales con reglas de integridad fuertes. PostgreSQL, MySQL, SQL Server, Oracle, ...
- **Documentales:** datos semiestructurados y esquemas flexibles. MongoDB, Couchbase, ...
- **Clave-valor:** caché, sesiones, contadores y colas simples. Redis, DynamoDB, ...
- **Grafos:** relaciones complejas entre objetos. Neo4j, JanusGraph, ...
- Otras familias **NoSQL** frecuentes
 - *Columna ancha:* Cassandra, HBase.
 - *Series temporales:* InfluxDB, TimescaleDB.
 - *Búsqueda/analítica:* Elasticsearch, OpenSearch.



Servicios gestionados

Una tendencia actual es consumir capacidades de backend como servicio gestionado.

- Ventajas
 - **Autenticación**, base de datos, almacenamiento, API y funciones serverless ya preparadas.
 - **Menos tiempo** de desarrollo y menos operaciones.
 - Ejemplos: Firebase, Supabase, Appwrite, AWS Amplify.
- Riesgos y límites
 - **Dependencia** del proveedor (vendor lock-in).
 - **Menor control** sobre arquitectura, rendimiento y costes a gran escala.
 - Necesidad de diseñar **portabilidad** desde el inicio.



Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Orígenes

Codd, E. F. A Relational Model of Data for Large Shared Data Banks. Communications of the ACM 13 (6): 377–387. [Link](#)

- Modelo sencillo y flexible basado en fundamentos matemáticos de teoría de conjuntos y lógica de predicados.
- Primeras implementaciones:
 - RDMS (MIT), primeros años de los 70
 - Ingres (Universidad de Berkeley), 1974
 - Oracle V2, 1979
 - IBM System R/DB2, 1979



¿Qué es una relación?

Una relación está compuesta por una **intensión** y por una **extensión**.

La **intensión** define la estructura de la relación: nombre, atributos, dominios y restricciones.

La intención cambia poco: cambia cuando cambia el diseño de la base de datos.

Por ejemplo, para una relación denominada `Empleados`, su intención podría ser:

- **nif**: Naturales de 8 dígitos seguidos de una letra mayúscula
- **nss**: Naturales de 12 dígitos
- **nombre**: Cadenas menores de 50 caracteres
- **edad**: Naturales menores o iguales a 200
- **salario**: Reales positivos
- **estadoCivil**: enumerado (soltero, casado, viudo, ...)

¿Qué es una relación?

La **extensión** es el conjunto de tuplas de la relación en un momento concreto.

La extensión cambia con las operaciones de inserción, actualización y borrado.

Por ejemplo, para la relación `Empleados`, su extensión podría ser:

```
-- Intensión
Empleados = { nif, nss, nombre, edad, salario, estadoCivil }

-- Extensión
Empleados = {
  ('12.345.678-Z', '123.456.789', 'Abel Abad', 21, 12000, 'Soltero'),
  ('23.456.789-D', '234.567.890', 'Braulio Brío', 32, 23000, 'Casado'),
  ('34.567.890-V', '345.678.901', 'Carlos Cepa', 43, 34000, 'Separado'),
  ('45.678.901-G', '456.789.012', 'David Díaz', 54, 45000, 'Divorciado'),
  ('56.789.012-B', '567.890.123', 'Enrique Estepa', 65, 56000, 'Casado')
}
```

Grado: número de atributos (columnas)

Cardinalidad: número de tuplas de la extensión (filas)

Implementación del concepto de relación

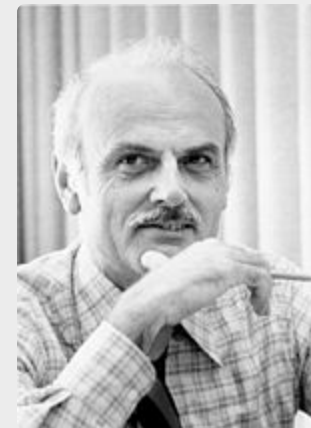
El concepto de relación se implementa mediante una **tabla**, cuyas **columnas** representan los atributos y las **filas** las tuplas.

Las diferencias con una relación son:

- Las filas están **ordenadas**, las tuplas no.
- Las filas pueden estar **repetidas**, las tuplas no.
- Las columnas (además de un nombre) tienen un **orden**, los atributos no.



Larry Ellison



Edgar F. Codd

¿Qué es un dominio?

Es el conjunto de valores admisibles que puede tomar un atributo de una relación.

Pueden ser:

- Un tipo básico sin restricciones: enteros, cadenas, ...
- Un tipo básico con restricciones: reales < 100 , cadenas de longitud ≤ 50 , naturales > 20 , ...
- Un tipo enumerado: lunes, martes, miércoles, ...
- Un tipo compuesto: fechas (dd/mm/aaaa), horas (hh:mm:ss), ...

Todos los dominios deben tener definido el operador de igualdad (=).

Algunos dominios tienen su propia álgebra con operadores $+$, $-$, $*$, $/$, ...

¿Qué es el valor nulo?

Si un atributo tiene valor nulo (null), significa que no se conoce su valor, que es desconocido.

El valor nulo puede asignarse a atributos definidos sobre cualquier dominio, pero no pueden compararse valores nulos de atributos definidos sobre dominios diferentes.

La introducción del valor nulo implica la necesidad de una lógica trivaluada:

A	B	A OR B	A AND B	NOT A
true	null	true	null	false
false	null	null	false	true
null	null	null	null	null

Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Superclaves - Unicidad

Una **superclave** es un conjunto de atributos que permite identificar de manera única las tuplas de una relación.

Pregunta práctica:

Si conozco estos atributos, ¿puedo distinguir una tupla de todas las demás?

Si la respuesta es sí, el conjunto de atributos cumple el criterio de **unicidad**.

La unicidad depende de la semántica del dominio, no sólo de los datos que haya cargados en este momento.

Clave Candidata

Una **clave candidata** es una superclave mínima.

Pregunta práctica:

¿Puedo quitar algún atributo y seguir identificando cada tupla?

Si la respuesta es sí, todavía no tengo una clave candidata: tengo una superclave con atributos sobrantes.

Las claves candidatas cumplen dos criterios:

- **unicidad**: identifican tuplas;
- **minimalidad**: no contienen atributos innecesarios.

Candidatas, Primarias y Alternativas

Una relación siempre tiene al menos una **clave candidata**, aunque puede tener varias.

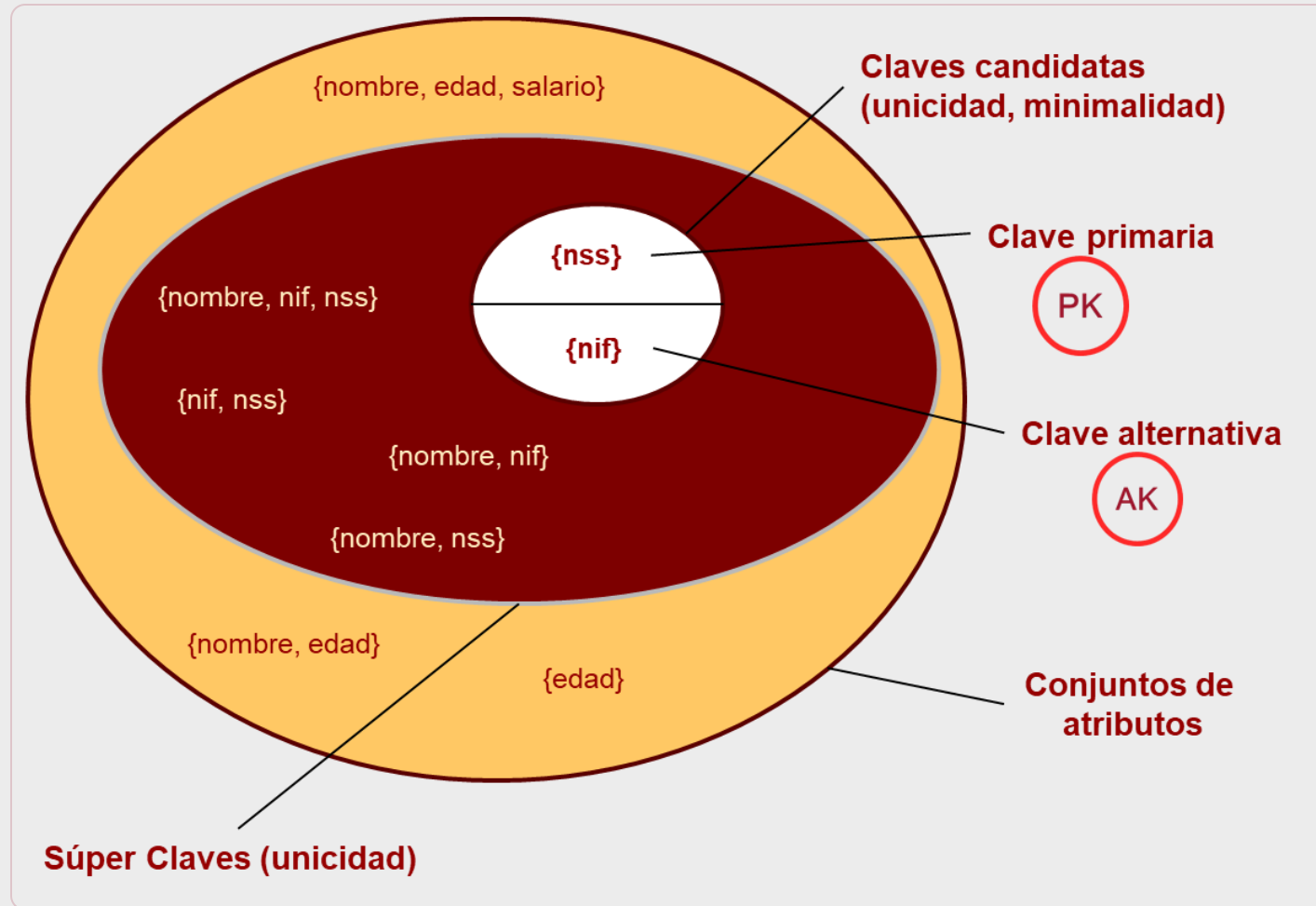
La **clave primaria (PK)** es la clave candidata que elegimos como identificador principal de la relación.

Las **claves alternativas (AK)** son las claves candidatas que no han sido elegidas como clave primaria.

La elección de la PK es una decisión de diseño: debe ser estable, sencilla de usar y no nula.



Relaciones entre claves



Claves ajenas (FK)

Una **clave ajena (FK)** es un conjunto de atributos de una relación cuyos valores deben coincidir con los de la clave primaria de otra relación.

Es la forma de representar las asociaciones del modelo conceptual en el modelo relacional.

Pregunta práctica:

¿Cómo sabe una tupla de esta relación con qué tupla de la otra relación está conectada?

Claves ajenas (FK)

Conviene usar el mismo nombre para la FK y para la PK a la que referencia.

```
CentrosSalud = {centroSaludId, dirección, teléfono}
    PK(centroSaludId)
CentrosSalud = {
    (1, 'Carretera de Carmona 48,      Sevilla', '954 954 954'),
    (2, 'Avenida de la Innovación 12, Sevilla', '955 123 456')
}

Personas = {personaId, nif, nss, nombre, centroSaludId}
    PK(personaId)
    FK(centroSaludId) / CentrosSalud
    AK(nif)
    AK(nss)
Personas = {
    (1, '28456319H', '281234567890', 'Ana Romero', 1),
    (2, '51789234M', '281234567891', 'Luis Ortega', 1)
}
```


Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Integridad de la entidad

Ningún atributo que forme parte de la clave primaria de una relación puede tomar el valor nulo.

Esta regla de integridad garantiza la identificación de tuplas mediante valores de la clave primaria.

```
Empleados = {nif, nss, nombre, edad, salario, estadoCivil}
              PK(nif)
              AK(nss)

Empleados = {
  ('12.345.678-Z', '123.456.789', 'Abel Abad', 21, 12000, 'soltero'),
  (null, '234.567.890', 'Braulio Brío', 32, 23000, 'casado'), -- viola regla
  ('34.567.890-V', '345.678.901', 'Carlos Cepa', 43, 34000, 'separado'),
  ('45.678.901-G', '456.789.012', 'David Díaz', 54, 45000, 'divorciado'),
  ('56.789.012-B', '567.890.123', 'Enrique Estepa', 65, 56000, 'casado')
}
```

El valor de una PK nunca puede ser nulo

Integridad referencial

Todos los atributos de una clave ajena deben tomar valores que coincidan con valores de la clave primaria correspondiente o bien tomar valores nulos.

Esta regla de integridad garantiza que todas las tuplas con claves ajenas se relacionan con otras tuplas existentes o bien con ninguna.

```
Empleados = {nss, nif, nombre, edad, centroSaludId}
  PK(nif)
  AK(nss)
  FK(centroSaludId) / CentrosSalud
Empleados = {
  ('123.456.789', '12.345.678-Z', 'Abel', 21, 48),
  ('234.567.890', '23.456.789-D', 'Braulio', 32, 1),
  ('345.678.901', '34.567.890-V', 'Carlos', 43, 2),
  ('456.789.012', '45.678.901-G', 'David', 40, 4),
  ('567.890.123', '56.789.012-B', 'Enrique', 65, null)
}
CentrosSalud = {centroSaludId, dirección, teléfono}
  PK(centroSaludId)
CentrosSalud = {
  (1, 'c/ Primera 10, Sevilla', '954 111 111'),
  (2, 'c/ Segunda 22, Sevilla', '954 222 222'),
  (3, 'c/ Tercera 8, Sevilla', '954 333 333'),
  (4, 'c/ Cuarta 15, Sevilla', '954 444 444')
}
```

El centro de salud con ID=48 no existe
Enrique no tiene asignado centro de salud

Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Calidad de los modelos relacionales

La calidad de un modelo relacional depende, entre otros factores, de las anomalías de manipulación que presente.

La forma de asegurar la calidad de un modelo relacional frente a las anomalías de manipulación es comprobar que está al menos en tercera forma normal (3FN).



Anomalías de manipulación

Supongamos una relación que contiene los datos de los inmuebles de una agencia de alquiler.

Cada inmueble tiene un código, una dirección, un precio de alquiler, una lista de propietarios con el porcentaje de propiedad del inmueble, y el código, nombre y cargo del empleado que lo gestiona.

```
Inmuebles = {inmuebleId, dirección, precio, propietarios, empleadoId, nombre, cargo}

Inmuebles = {
  ('10A', 'C/ Norte, 15', 600, 'P. Río, 70% / D. Páez, 30%', 3, 'S. Blas', 'Resp. Zona'),
  ('30A', 'C/ Sur, 15', 500, 'E. Ruz, 100%', 3, 'S. Díaz', 'Resp. Zona'),
  ('87B', 'C/ Este, 8', 700, 'R. Bas, 50% / P. Río, 50%', 5, 'N. Clos', 'Resp. Zona'),
  ('91A', 'C/ Oeste, 10', 650, 'M. Gil, 40% / M. Quer, 60%', 8, 'G. Peña', 'Comercial'),
  ('23B', 'C/ Sol, 14', 800, 'R. Mel, 70% / J. Val, 30%', 8, 'G. Peña', 'Comercial')
}
```

¿Qué problemas presenta la relación?

Datos redundantes: el nombre y el cargo de cada empleado se repiten tantas veces como inmuebles gestione, malgastando espacio.

Riesgos de incoherencia: la redundancia de datos implica el riesgo de que se vuelvan incoherentes si no se actualizan todas las ocurrencias a la vez.

Anomalías de inserción: hasta que un empleado no gestione un inmueble no se puede registrar en el sistema de información.

Anomalías de actualización: si un empleado cambia de cargo hay que actualizarlo múltiples veces en lugar de hacerlo una sola vez.

Anomalías de eliminación: si un empleado deja de gestionar inmuebles, sus datos desaparecen del sistema de información.

Problemas de consulta: ¿cómo se podrían conocer todos los inmuebles de un determinado propietario?

¿Qué problemas presenta la relación?

¿Cuántos problemas de los anteriores se evitan con el nuevo modelo relacional de tres relaciones?

```
-- Intensión
Empleados = {empleadoId, nombre, cargo }
             PK(empleadoId)
Inmuebles = {inmuebleId, dirección, precio, empleadoId}
             PK(inmuebleId)
             FK(empleadoId) / Empleados
Propietarios = {inmuebleId, propietario, porcentaje}
                PK(inmuebleId, propietario)
                FK(inmuebleId) / Inmuebles
```

```
-- Extensión
Inmuebles = {
  ('10A', 'C/ Norte, 15', 600, 3),
  ('30A', 'C/ Sur, 15', 500, 3),
  ('87B', 'C/ Este, 8', 700, 5),
  ('91A', 'C/ Oeste, 10', 650, 8),
  ('23B', 'C/ Sol, 14', 800, 8)
}
Empleados = {
  (3, 'S. Blas', 'Resp. Zona'),
  (5, 'N. Clos', 'Resp. Zona'),
  (8, 'G. Peña', 'Comercial')
}
Propietarios = {
  ('10A', 'P. Río', 70),
  ('10A', 'D. Páez', 30),
  ('30A', 'E. Ruz', 100),
  ('87B', 'R. Bas', 50),
  ('87B', 'P. Río', 50),
  ('91A', 'M. Gil', 40),
  ('91A', 'M. Quer', 60),
  ('23B', 'R. Mel', 70),
  ('23B', 'J. Val', 30)
}
```


Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Notación

$I(R)$: intensión de R (conjunto de atributos).

$E(R)$: extensión de R (conjunto de tuplas).

$t_1, t_2 \in E(R)$: dos tuplas de R .

$t_1[X]$: proyección de la tupla t_1 sobre el conjunto de atributos X (valores de X en t_1).

$t_1[Y]$: proyección de la tupla t_1 sobre Y .



¿Qué es una dependencia funcional?

Decimos que X determina funcionalmente a Y cuando, al conocer X , el valor de Y queda fijado.

Se denota como $X \rightarrow Y$.

Ejemplos:

- si conozco `inmuebleId`, conozco `dirección` ;
- si conozco `empleadoId`, conozco `nombre` ;
- si conozco `empleadoId`, conozco `cargo` .

En otras palabras: no pueden existir dos tuplas con el mismo valor de X y distinto valor de Y .

¿Qué es una dependencia funcional?

Si R es una relación y X e Y son dos subconjuntos de los atributos de R , se dice que X **determina funcionalmente a Y** o que Y **depende funcionalmente de X** , si y sólo si **siempre** que dos tuplas tienen los mismos valores de X , también tienen los mismos valores de Y .

$$\forall t_1, t_2 \in E(R) \cdot ((t_1[X] = t_2[X]) \Rightarrow (t_1[Y] = t_2[Y]))$$

En otras palabras: **nunca** dos tuplas con los mismos valores de X pueden tener distintos valores de Y .

$$\nexists t_1, t_2 \in E(R) \cdot ((t_1[X] = t_2[X]) \wedge (t_1[Y] \neq t_2[Y]))$$

¿Cómo se identifican las dependencias funcionales?

Las dependencias funcionales **no pueden deducirse de los datos** de la extensión de una relación.

Sólo podría descartarse su existencia si los datos de la extensión las contradijeran.

Por lo tanto:

Las dependencias funcionales dependen de la semántica de los atributos de las relaciones en el modelo conceptual y, por extensión, en el dominio del problema.

Dependencias del ejemplo

En la relación **Inmuebles** aparecen dependencias funcionales como estas:

inmuebleId → *dirección*
inmuebleId → *precio*
inmuebleId → *empleadoId*
empleadoId → *nombre*
empleadoId → *cargo*

Estas dependencias no salen de mirar las filas, salen de entender qué significa cada atributo.

```
Inmuebles = {inmuebleId, dirección, precio, propietarios, empleadoId, nombre, cargo}
             PK(inmuebleId)

Inmuebles = {
  ('10A', 'C/ Norte, 15', 600, 'P. Río, 70% / D. Páez, 30%', 3, 'S. Blas', 'Resp. Zona'),
  ('30A', 'C/ Sur, 15', 500, 'E. Ruz, 100%', 3, 'S. Díaz', 'Resp. Zona'),
  ('87B', 'C/ Este, 8', 700, 'R. Bas, 50% / P. Río, 50%', 5, 'N. Clos', 'Resp. Zona'),
  ('91A', 'C/ Oeste, 10', 650, 'M. Gil, 40% / M. Quer, 60%', 8, 'G. Peña', 'Comercial'),
  ('23B', 'C/ Sol, 14', 800, 'R. Mel, 70% / J. Val, 30%', 8, 'G. Peña', 'Comercial')
}
```

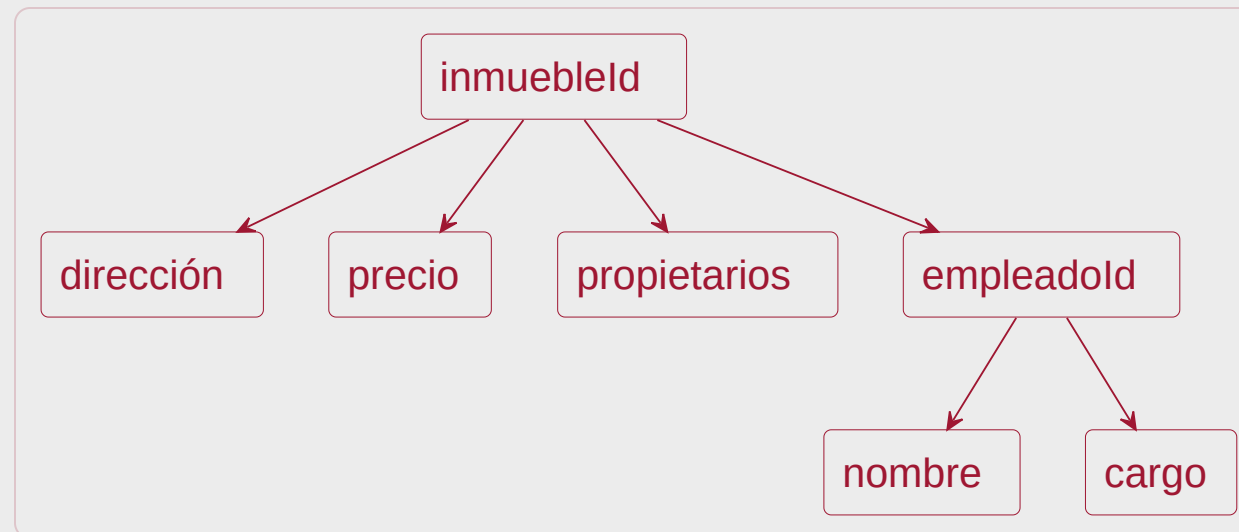
Grafo de dependencias funcionales

Forma gráfica de representar las dependencias funcionales de un modelo relacional.

Los **nodos** son **atributos** o conjuntos de atributos.

Los **arcos** son las **dependencias funcionales**.

Normalmente sólo se representan dependencias funcionales que determinan a un solo atributo.



Contenido

Introducción

Conceptos básicos

Claves

Integridad

Anomalías de manipulación

Dependencias funcionales

Formas normales



Formas normales

Las formas normales son reglas para comprobar que una relación no mezcla información que debería estar en relaciones distintas.

Cada forma normal elimina un tipo de problema:

Forma normal	Pregunta que debemos hacernos
1FN	¿Cada celda contiene un único valor?
2FN	¿Cada atributo depende de toda la clave?
3FN	¿Cada atributo depende sólo de la clave?

Cada FN incluye a la anterior: si una relación está en 3FN, también está en 2FN y en 1FN.

Primera forma normal (1FN)

Una relación está en 1FN cuando cada atributo de cada tupla contiene **un único valor**.

La 1FN evita:

- listas dentro de una celda;
- atributos repetidos como `teléfono1`, `teléfono2`, `teléfono3`;
- valores que habría que partir para poder consultarlos bien.

```
-- 1FN OK
Clientes = {clienteId, nombre, teléfono}
          PK(clienteId, teléfono)

Clientes = {
  (1, 'Abel Abad', '666111222'),
  (2, 'Braulio Brío', '666222333')
}
```

```
-- 1FN FAIL
Clientes = {clienteId, nombre, teléfonos}
          PK(clienteId)

Clientes = {
  (1, 'Abel Abad', '666111222'),
  (2, 'Braulio Brío', '666222333 / 666555666')
}
```

Ejemplo 1FN - Clientes

Queremos permitir varios teléfonos por cliente.

La solución no es añadir más columnas, porque no sabemos cuántos teléfonos puede tener cada cliente.

```
Clientes = {clienteId, nombre, teléfono}
           PK(clienteId)
```

```
Clientes ={
  (1, 'Abel Abad', '666111222'),
  (2, 'Braulio Brío', '666222333'),
  (3, 'Carlos Cepa', '666333444')
}
```

-- Mala solución: límite artificial de teléfonos

```
Clientes = {clienteId, nombre, teléfono1, teléfono2, teléfono3}
           PK(clienteId)
```

```
Clientes ={
  (1, 'Abel Abad', '666111222', null, null),
  (2, 'Braulio Brío', '666222333', '666555666', '954456789'),
  (3, 'Carlos Cepa', '666333444', '954123123', null)
}
```

Ejemplo 1FN - Clientes

La solución correcta es separar los datos del **cliente** de los datos de sus **teléfonos**.

Así cada teléfono ocupa una tupla distinta.

```
Clientes = {clienteId, nombre}
           PK(clienteId)
Clientes = {
  (1, 'Abel Abad'),
  (2, 'Braulio Brío'),
  (3, 'Carlos Cepa')
}

Teléfonos = {clienteId, teléfono}
           PK(clienteId, teléfono)
           FK(clienteId) / Clientes
Teléfonos = {
  (1, '666111222'),
  (2, '666222333'),
  (2, '666555666'),
  (2, '954456789'),
  (3, '666333444'),
  (3, '954123123')
}
```

Ejemplo 1FN - Inmuebles

Un atributo en plural suele avisar de un problema, probablemente guarda varios valores.

```
Inmuebles = {inmuebleId, dirección, precio, propietarios, empleadoId, nombre, cargo}
PK(inmuebleId)
Inmuebles = {
  ('10A', 'C/ Norte, 15', 600, 'P. Río, 70% / D. Páez, 30%', 3, 'S. Blas', 'Resp. Zona'),
  ('30A', 'C/ Sur, 15', 500, 'E. Ruz, 100%', 3, 'S. Díaz', 'Resp. Zona'),
  ('87B', 'C/ Este, 8', 700, 'R. Bas, 50% / P. Río, 50%', 5, 'N. Clos', 'Resp. Zona'),
  ('91A', 'C/ Oeste, 10', 650, 'M. Gil, 40% / M. Quer, 60%', 8, 'G. Peña', 'Comercial'),
  ('23B', 'C/ Sol, 14', 800, 'R. Mel, 70% / J. Val, 30%', 8, 'G. Peña', 'Comercial')
}
```

Para cumplir 1FN, cada propietario debe ser un valor independiente.

```
Inmuebles = {inmuebleId, dirección, precio, propietario, porcentaje, empleadoId, nombre, cargo}
PK(inmuebleId, propietario)
Inmuebles = {
  ('10A', 'C/ Norte, 15', 600, 'P. Río', 70, 3, 'S. Blas', 'Resp. Zona'),
  ('10A', 'C/ Norte, 15', 600, 'D. Páez', 30, 3, 'S. Blas', 'Resp. Zona'),
  ('30A', 'C/ Sur, 15', 500, 'E. Ruz', 100, 3, 'S. Díaz', 'Resp. Zona'),
  ('87B', 'C/ Este, 8', 700, 'R. Bas', 50, 5, 'N. Clos', 'Resp. Zona'),
  ('87B', 'C/ Este, 8', 700, 'P. Río', 50, 5, 'N. Clos', 'Resp. Zona'),
  ('91A', 'C/ Oeste, 10', 650, 'M. Gil', 40, 8, 'G. Peña', 'Comercial'),
  ('91A', 'C/ Oeste, 10', 650, 'M. Quer', 60, 8, 'G. Peña', 'Comercial'),
  ('23B', 'C/ Sol, 14', 800, 'R. Mel', 70, 8, 'G. Peña', 'Comercial'),
  ('23B', 'C/ Sol, 14', 800, 'J. Val', 30, 8, 'G. Peña', 'Comercial')
}
```

Segunda forma normal (2FN)

Una relación está en 2FN cuando está en 1FN y ningún atributo no clave depende sólo de una parte de una clave compuesta.

La pregunta práctica es:

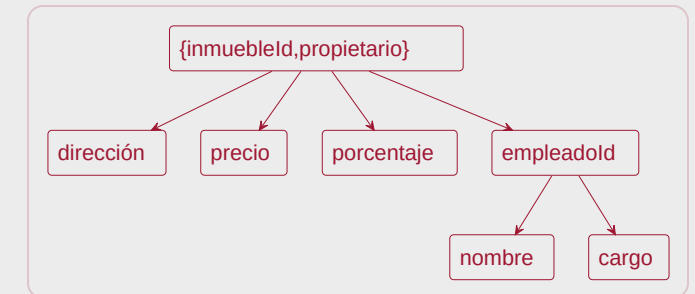
Si quito una parte de la clave, ¿puedo seguir sabiendo el valor de este atributo?

Si la respuesta es sí, hay una **dependencia parcial**.

La 2FN obliga a separar esos datos en otra relación.

Ejemplo 2FN - Inmuebles

```
Inmuebles = { inmuebleId, dirección, precio, propietario,  
              porcentaje, empleadoId, nombre, cargo }  
PK(inmuebleId, propietario)  
Inmuebles = {  
  ('10A', 'C/ Norte, 15', 600, 'P. Río', 70, 3, 'S. Blas', 'Resp. Zona'),  
  ('10A', 'C/ Norte, 15', 600, 'D. Páez', 30, 3, 'S. Blas', 'Resp. Zona'),  
  ('30A', 'C/ Sur, 15', 500, 'E. Ruz', 100, 3, 'S. Díaz', 'Resp. Zona'),  
  ('87B', 'C/ Este, 8', 700, 'R. Bas', 50, 5, 'N. Clos', 'Resp. Zona'),  
  ('87B', 'C/ Este, 8', 700, 'P. Río', 50, 5, 'N. Clos', 'Resp. Zona'),  
  ('91A', 'C/ Oeste, 10', 650, 'M. Gil', 40, 8, 'G. Peña', 'Comercial'),  
  ('91A', 'C/ Oeste, 10', 650, 'M. Quer', 60, 8, 'G. Peña', 'Comercial'),  
  ('23B', 'C/ Sol, 14', 800, 'R. Mel', 70, 8, 'G. Peña', 'Comercial'),  
  ('23B', 'C/ Sol, 14', 800, 'J. Val', 30, 8, 'G. Peña', 'Comercial')  
}
```



**dirección y
precio** dependen
sólo de **inmuebleId**,
no de toda la clave.

Ejemplo 2FN - Inmuebles

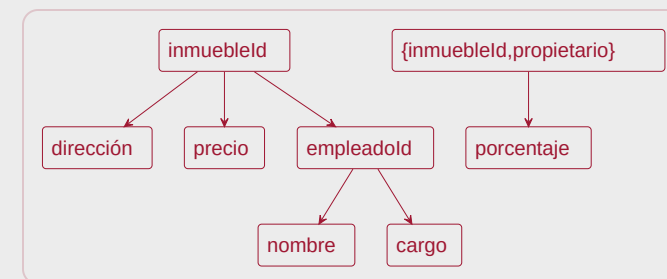
Descomposición en relaciones más simples:

```
Inmuebles = { inmuebleId, dirección, precio, empleadoId, nombre, cargo }  
            PK(inmuebleId)
```

```
Inmuebles = {  
  ('10A', 'C/ Norte, 15', 600, 3, 'S. Blas', 'Resp. Zona'),  
  ('30A', 'C/ Sur, 15', 500, 3, 'S. Díaz', 'Resp. Zona'),  
  ('87B', 'C/ Este, 8', 700, 5, 'N. Clos', 'Resp. Zona'),  
  ('91A', 'C/ Oeste, 10', 650, 8, 'G. Peña', 'Comercial'),  
  ('23B', 'C/ Sol, 14', 800, 8, 'G. Peña', 'Comercial')  
}
```

```
Propietarios = { inmuebleId, propietario, porcentaje }  
                PK(inmuebleId, propietario)  
                FK(inmuebleId) / Inmuebles
```

```
Propietarios = {  
  ('10A', 'P. Río', 70),  
  ('10A', 'D. Páez', 30),  
  ('30A', 'E. Ruz', 100),  
  ('87B', 'R. Bas', 50),  
  ('87B', 'P. Río', 50),  
  ('91A', 'M. Gil', 40),  
  ('91A', 'M. Quer', 60),  
  ('23B', 'R. Mel', 70),  
  ('23B', 'J. Val', 30)  
}
```



Ahora **porcentaje** sí depende de toda la clave **{inmuebleId, propietario}**.

Regla general para la 2FN

Si una relación tiene una clave compuesta, cada atributo no clave debe depender de la clave completa.

```
-- Relación original
R = { K1, K2, X, Y }
    PK(K1, K2)

-- Dependencias funcionales
K1      -> X
{K1,K2} -> Y
```

R no está en 2FN porque X depende sólo de K1 .

```
-- Descomposición en 2FN
R1 = { K1, X }
    PK(K1)

R2 = { K1, K2, Y }
    PK(K1, K2)
```

Tercera forma normal (3FN)

Una relación está en 3FN cuando está en 2FN y ningún atributo no clave depende de otro atributo no clave.

La idea clásica:

Los atributos deben depender de la clave, de toda la clave y de nada más que la clave.

La 3FN elimina **dependencias transitivas**:

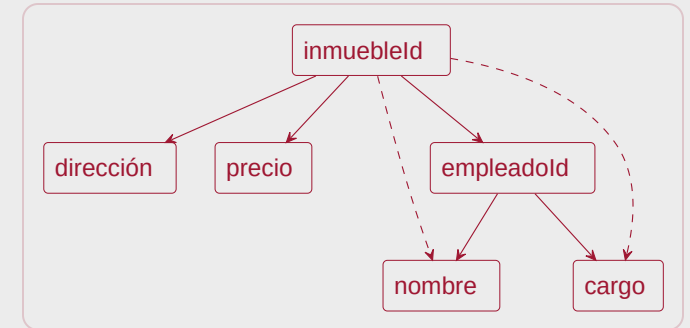
clave $\rightarrow$ atributo no clave $\rightarrow$ otro atributo no clave

Ejemplo 3FN - Inmuebles

Aunque ya no hay dependencias parciales, siguen apareciendo datos del empleado dentro de Inmuebles .

```
Inmuebles = { inmuebleId, dirección, precio, empleadoId, nombre, cargo }  
PK(inmuebleId)
```

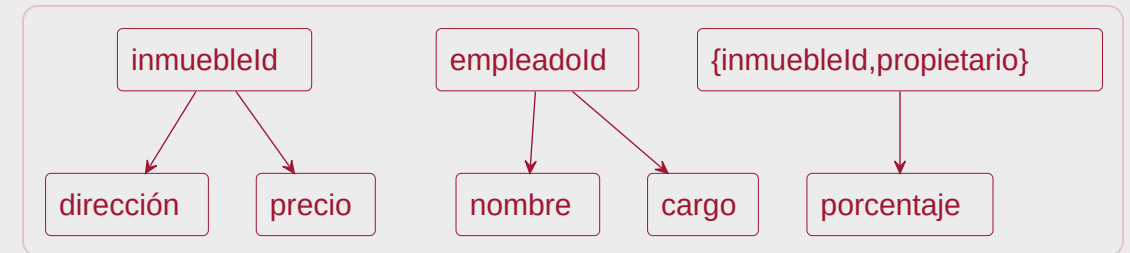
nombre y cargo no describen al inmueble, describen al empleado.



Ejemplo 3FN - Inmuebles

La solución es separar los datos de **inmuebles** y **empleados**.

```
Inmuebles = { inmuebleId, dirección, precio, empleadoId }  
            PK(inmuebleId)  
            FK(empleadoId) / Empleados  
  
Empleados = { empleadoId, nombre, cargo }  
            PK(empleadoId)
```



Cada atributo queda en la relación a la que pertenece.

Normalización y transformación

La normalización ayuda a detectar problemas en relaciones mal diseñadas. El siguiente paso será obtener relaciones bien estructuradas desde el modelo conceptual:

- entidades $\rightarrow$ relaciones;
- asociaciones 1:N $\rightarrow$ claves ajenas;
- asociaciones M:N $\rightarrow$ relaciones auxiliares;
- atributos multivaluados $\rightarrow$ relaciones separadas;
- generalizaciones $\rightarrow$ estrategias de transformación.



El fallo estaba en el modelo

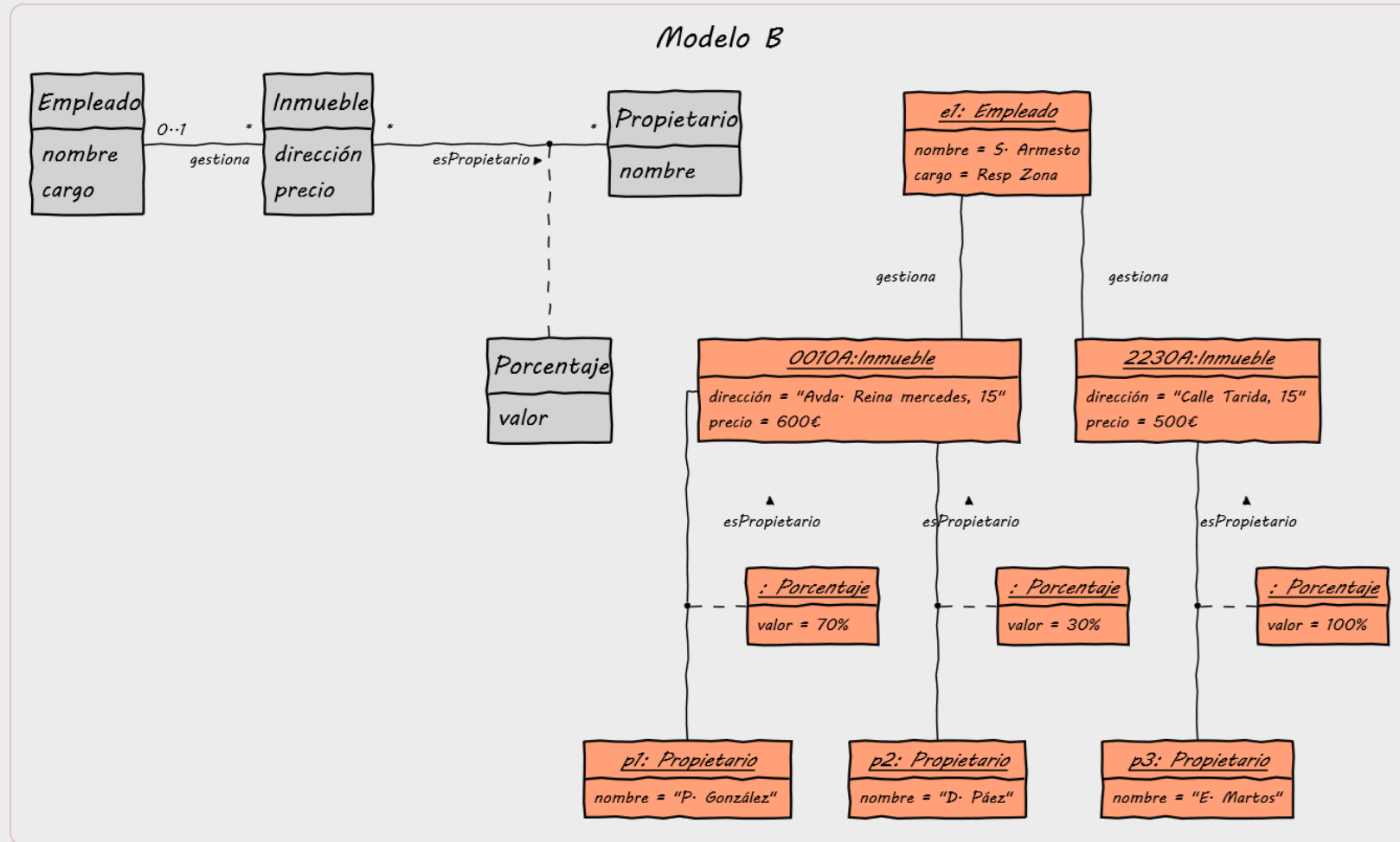
Modelo A

Inmueble
dirección
precio
propietarios
nombre
cargo

<u>0010A:Inmueble</u>
dirección = "Avda. Reina mercedes, 15"
precio = 600€
propietarios = {"P. González", 30%}, {"D. Páez", 30%}
nombre = S. Armesto
cargo = Resp. Zona

<u>2230A:Inmueble</u>
dirección = "Calle Tarida, 15"
precio = 500€
propietarios = {"E. Martos", 100%}
nombre = S. Armesto
cargo = Resp. Zona

El fallo estaba en el modelo



Conclusiones

Una **base de datos** organiza datos del mundo real para soportar procesos de **consulta y gestión**.

El **modelo relacional** representa la información mediante **relaciones, tuplas y atributos**.

La calidad del modelo depende de su capacidad para evitar **redundancia y anomalías de manipulación**.

El diseño relacional debe mantener **trazabilidad** con los requisitos y con el modelo conceptual previo.

Conclusiones

Las **claves candidatas** permiten identificar unívocamente tuplas; la **clave primaria** es una de ellas.

Las **claves ajenas (FK)** conectan relaciones y hacen explícitas las asociaciones del dominio.

La **integridad de entidad** exige PK no nula y sin duplicados.

La **integridad referencial** obliga a que toda FK apunte a una PK existente (o sea nula si procede).

Conclusiones

Las **dependencias funcionales** capturan restricciones semánticas entre atributos.

La **1FN** elimina grupos repetidos y obliga a valores **atómicos**.

La **2FN** elimina dependencias **parciales** respecto a claves compuestas.

La **3FN** elimina dependencias **transitivas** y ayuda a obtener modelos más consistentes.

El siguiente paso será obtener esos modelos de forma sistemática a partir del modelo conceptual.





Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA